



Optimising Autonomous Vehicle Task Offloading

A Prescriptive Analytics Approach

Prepared by:

Lee Pei Wen	U2410105A
Pahwa Ronak	U2222098K
Lewis Dang	N2504171J
Sheng Xiayi	U2222867A

Introduction

Business Problem

Assumptions

Methodology

Model #1 Minimize Latency

Model #2 Minimize Daily Costs for Auto Manufacturers

Analysis of Solution

Implementation Feasibility

Outcome Evaluation

Limitations

Recommendations

Conclusion

References

Introduction

The Autonomous Vehicles (AVs) market is one of the most rapidly evolving transportation and artificial intelligence industries, backed up by the continuous advancements in technology such as machine learning, computing infrastructure and sensor technologies. By 2034, the global autonomous vehicle market is expected to grow significantly, and is expected to reach USD 80 billion in revenue (Fortune Business Insights, 2026). This rapid expansion highlights the increasing potential and importance for an intelligent, safe and automated transportation system.

AVs, also known as self-driving vehicles, rely on complex sensing systems and data collected from different sources such as LiDAR, radar and ultrasonic sensors to perceive the environment and make real-time driving operations (Viktor & Kiss, 2026). AVs continuously process these large amounts of data to help them make decisions on tasks required for safe and efficient driving (Viktor & Kiss, 2026). These tasks often differ in terms of their urgency and complexity. For example, pedestrian detection is highly time-sensitive and critical for pedestrian safety, while traffic sign recognition is important but less critical when compared to pedestrian detection. On the other hand, scene understanding is less time-sensitive and complex.

These tasks often occur simultaneously on the road, but they require different levels of speed and processing power. Hence, all of the tasks are competing for limited computational resources, leading to a situation whereby AVs must efficiently manage how these tasks are processed while maintaining overall performance. Maintaining performance is key as they affect the safety and reliability of AVs on the road.

To handle these requirements, tasks can be processed using two approaches: local processing and cloud processing (Aleksandra, 2026). Local processing involves executing tasks directly within the vehicle using onboard computation resources, which has minimal latency whereby the process speed is only limited by onboard compilation complexity. However, it has higher upfront costs as these powerful onboard systems have to be installed in the car. On the other hand, cloud processing involves offloading tasks to external servers that have higher computational power. This allows the system to have the ability to handle complex tasks without installing expensive hardware, lowering upfront costs for these items. However, this introduces network latency due to data transmission and it requires ongoing operational costs such as transfer and cloud computing fees.

Business Problem

In practice, many computing systems have gradually started to adopt the hybrid approach that combines both local and cloud processing. However, these allocation is often based on predefined rules rather than optimisation, which may not fully account for the variation in tasks requirement and competing factors, leading to inefficient resource utilisation, higher overall costs and suboptimal performance.

The key to processing these tasks efficiently is to balance 3 key trade-offs: efficiency, reliability and costs. Increasing onboard processing reduces reliance on onboard processing and it prevents any network latency, but it introduces costs. At the same time, critical tasks must also be processed reliably within time limits. Hence, based on these key trade-offs and limitations, this report aims to

develop a systematic approach to task allocation, using Integer Linear Programme (ILP) to determine the optimal allocation of tasks, and to provide practical and actionable business recommendations.

Specifically, our main objective is to determine the optimal allocation of tasks between local and cloud processing in order to minimize total costs, while satisfying system constraints such as latency, computational capacity and safety requirements.

Assumptions

- 1) In reality, tasks may be interdependent. However for this project, each task is assumed to be independent of one other and can be processed individually. There are no dependencies between tasks, and each task requires a single processing decision. This assumption allows each task to be evaluated individually.
- 2) Each task is assumed to be processed entirely local or in the cloud. This assumption allows for the formulation of binary decision variables.
- 3) In real-world systems, factors such as latency and costs are constantly changing due to network conditions, environmental factors, etc. For this project, latency and costs are assumed to be known and constant for each task. This allows for an optimal optimization model without accounting for the small and insignificant changes.

Methodology

To identify the typical task workload distribution for the study, a vehicular dataset of realistic car status on major US urban roadways (simulated with VEINS OMNeT++ framework) is used (Qayyum, 2025). The original dataset contains data rows from multiple simulated vehicles. However, in this study we filter data from the car with VehicleID=0 to get 8,640 rows of records. For every experiment, a stratified sampling of 800 tasks based on the TaskType field will be used as an one-day typical workload for the LOP.

From the dataset, we have extracted the following fields to construct our model: *TaskSize* (computational requirement of a task, ranging from 1 to 10), *CPU_Available* (currently available local system resources), *NetworkLatency* (the time needed for communicating from the car to the cloud server), *SignalStrength* (the wireless connectivity of the car), *TaskPriority* (High/Medium/Low, mapped to the model input parameter *latency_limit*, indicating the time constraints to finish a task), and *WeatherCondition*.

A binary safety constraint *force_local* is constructed based on *SignalStrength* and *WeatherCondition*, to be more specific, a task will be flagged *force_local*=1 to be executed locally in the case of weak signals or stormy weather, covering ~10.9% of the dataset.

Two integer linear programming models are then proposed. One to minimize the latency with a given hardware setting, and the other to find the most effective setting for the manufacturer to minimize the cost for vehicle production and operations.

Both models are solved using the Gurobi solver. With a sample size of 800 tasks, both could be solved within 1 second.

Model #1 Minimize Latency

The first model sets its linear optimization goal to minimize the total computation latency by making a binary decision on executing each task either locally or on the cloud.

Decision variable:

$$x_i \in \{0, 1\}$$

of which 0 representing local execution and 1 representing cloud computing.

Object function:

$$\min \sum_{i \in \mathcal{I}} [(1 - x_i) \cdot c_i^{\text{local}} + x_i \cdot c_i^{\text{cloud}}]$$

of which $c_i^{\text{local}} = \text{TaskSize}_i / (\text{CPU_Available}_i / 100)$ estimating the task execution time based on available system resources, and $c_i^{\text{cloud}} = \text{NetworkLatency}_i \times c_{\text{cloud}}$ estimating the cloud computing latency based on current network latency and a calibration factor, set $c_{\text{cloud}} = 1$.

Constraint 1 (Safety):

$$x_i \leq 1 - f_i$$

of which f_i represents the *force_local* variable for task i, if $f_i = 1$, the task is forced to be computed locally.

Constraint 2 (Computing Power Limit):

$$\sum_{i \in \mathcal{I}} (1 - x_i) \cdot s_i \leq \text{CPU_CAP}$$

of which s_i is the TaskSize of task i , CPU_CAP is the hypothetical local computing power limit, CPU_CAP = mean(CPU_Available) $\times$ buffer $\times$ N / 10 with default buffer = 0.8.

Baseline comparison:

To compare the model performance, we have constructed two greedy strategy baselines: Greedy-Local (always prioritize local execution, only compute on the cloud while local is overloaded) and Greedy-Cloud (always prioritize cloud execution, only leave tasks with force_local = 1 to compute locally).

Model #2 Minimize Daily Costs for Auto Manufacturers

The constraint of the first model is only optimizing the latency without considering the monetary cost to do so. In reality, the onboard hardware is a one-time capital expense for the auto manufacturers, which suggests its marginal costs would be near-zero once purchased. In contrast, cloud execution incurs pay-per-use operational costs. For this reason, we reconstruct the second model with the above-mentioned two costs in order to minimize the monetary costs for auto manufacturers while choosing different inference hardware platforms on their cars.

Costs:

- **Per-Day Average Capital Expenses:**

$$\text{Daily_CapEx} = H \cdot b / T$$

of which H is the price of the baseline hardware which could just allow all tasks to run locally, default to SGD 5,000. b representing the performance of the selected hardware proportioned to the baseline. T being the expected lifespan of the vehicle, default to 3,650 days.

- **Cloud Per-Task Operational Expenses:**

$$c_i^{\text{net}} = s_i \cdot \text{congestion}_i \cdot P_{\text{transfer}} + s_i \cdot P_{\text{cloud_cpu}}$$

of which congestion = NetworkLatency / max(NetworkLatency), P representing the monetary cost for transferring and computing the task on the cloud, both default to 0.001. The resulting cost is around SGD 0.0084/task.

Decision variable:

$$x_i \in \{0, 1\}$$

of which 0 representing local execution and 1 representing cloud computing.

Object function:

$$\min \quad \frac{H \cdot b}{T} + \sum_{i \in \mathcal{I}} x_i \cdot c_i^{\text{net}}$$

As $\text{CapEx} = H \cdot b / T$ is a constant number for a given buffer b , the solver is actually minimizing the OpEx, and then add CapEx back for total cost estimation.

Constraint 1 (Safety, unchanged):

$$x_i \leq 1 - f_i$$

of which f_i represents the force_local variable for task i , if $f_i = 1$, the task is forced to be computed locally.

Constraint 2 (Computing Power Limit, unchanged):

$$\sum_{i \in \mathcal{I}} (1 - x_i) \cdot s_i \leq \text{CPU_CAP}$$

of which s_i is the TaskSize of task i , CPU_CAP is the hypothetical local computing power limit at a given buffer level, $\text{CPU_CAP} = \text{mean}(\text{CPU_Available}) \times b \times N / 10$ with default buffer $b = 0.8$.

Constraint 3 (Latency SLA, new):

$$(1 - x_i) \cdot l_i^{\text{local}}(b) + x_i \cdot l_i^{\text{cloud}} \leq L_i + M \cdot v_i$$

this is to ensure that all tasks meet the Service Level Agreement (SLA) latency performance requirements, of which $l_i^{\text{local}}(b) = \text{base_latency}_i \times (\text{CPU_CAP_BASE} / \text{CPU_CAP}(b))$ allows tasks to run faster on a processor with better performance / larger buffer b , L_i is the SLA limit for the three task priorities, default to High = 80ms, Medium = 120ms, Low = 200ms. M being the relaxation coefficient, allowing a small number of tasks to violate the latency requirements, while $v_i \in \{0, 1\}$ is its indicative variable, also used in **Constraint 4**.

Constraint 4 (SLA Violation, new):

$$\sum_{i \in \mathcal{I}} v_i \leq V_{\max}$$

of which $V_{\max}$ is default to 0.5% of the number of tasks with force_local = 1, with a minimum of 1.

Baseline comparison:

The same Greedy-Local strategy for a given buffer b is kept in Model #2 for comparison. However the primary goal of Model #2 is its sensitivity analysis with different b values representing different options of hardware choices.

Analysis of Solution

This section evaluates whether the integer linear programming model achieves the report's objective of minimising daily processing cost while still satisfying the safety, capacity, and latency constraints defined earlier. At the baseline configuration of buffer = 0.8 and $N = 800$ tasks, the ILP reduces total daily cost from SGD 2.2575 under Greedy-Local to SGD 2.0732, which is a 8.2% improvement. Both strategies assume the same onboard hardware, so their daily CapEx is identical at SGD 1.0959. The saving therefore comes entirely from better operating decisions: ILP lowers cloud OpEx from SGD 1.1616 to SGD 0.9773 per day while assigning 641 tasks locally and 159 to the cloud. The solution also remains operationally controlled, with only 1 SLA violation out of 800 tasks, which is still below

the allowed limit of 4. This shows that the cost reduction is not created by spending more on hardware, but by making smarter offloading choices with the same hardware budget.

The reason ILP outperforms the heuristic baseline is that the two methods make decisions differently. Greedy-Local is a sequential overflow rule: it keeps tasks local until the processor is full, and then offloads the remaining tasks without considering which cloud assignments are cheap or expensive. By contrast, the ILP evaluates all 800 tasks jointly and chooses the subset to offload that minimises total cost subject to the force_local safety rule, the local CPU capacity limit, and the latency-SLA requirement. In practice, this means scarce local capacity is reserved for tasks that are expensive to send to cloud or are more constrained by signal quality and latency, while cheaper tasks are selectively offloaded. This is the core prescriptive analytics advantage of the model: global optimisation produces better decisions than a local rule even when both methods operate under the same resource envelope.

Scenario	Greedy-Local (SGD/day)	ILP (SGD/day)	Cloud offload (%)	Savings (%)
Low priority	1.6385	1.4987	21.3	8.5
Medium priority	1.4383	1.3982	19.8	2.8
High priority	1.3658	1.3476	15.9	1.3
Low traffic	1.5868	1.5127	24.8	4.7
Medium traffic	1.4765	1.4144	17.3	4.2
High traffic	1.3726	1.3167	16.1	4.1
Full sample	2.2313	2.0515	19.5	8.1

Table 5.1. Cost comparison across the seven tested scenarios in the final notebook run

Table 5.1 shows that the ILP dominates Greedy-Local in every tested scenario. Savings range from 1.3% in high priority to 8.5% for the low-priority subset, with the full sample still achieving a material 8.1% reduction. This is important because it indicates that the solution is not overfitted to a single workload composition. Instead, the optimisation advantage remains visible across both traffic-density and task-priority shifts, which is exactly the type of robustness required for an AV deployment environment where workload composition changes continuously.

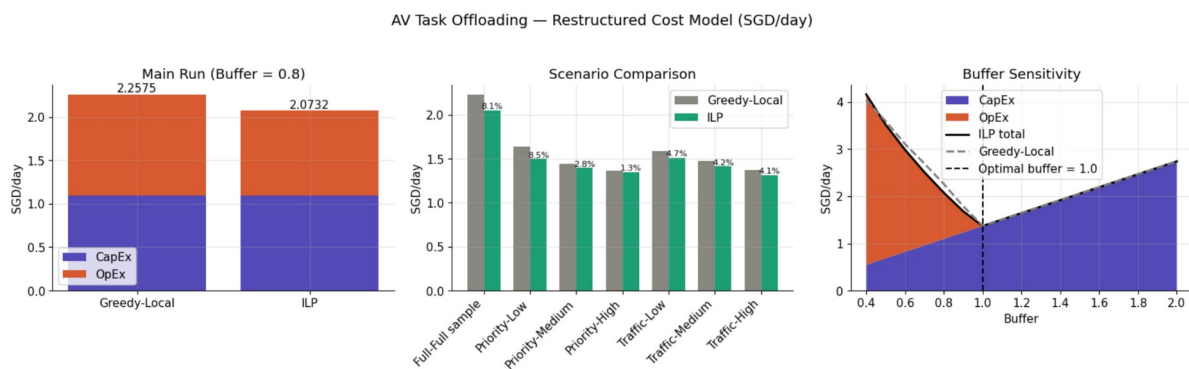


Figure 5.1. Baseline cost breakdown, scenario-wise comparison, and the CapEx-OpEx trade-off

The buffer sensitivity analysis adds another managerial interpretation layer. As the processor buffer increases, daily CapEx rises almost linearly because more onboard computing power is being purchased. At the same time, OpEx falls sharply because fewer tasks need to be pushed to the cloud. Within the feasible operating range of the final notebook run, ILP remains better than Greedy-Local at buffers 0.7, 0.8 and 0.9, delivering savings between 5.97% and 8.16%. The total daily cost is

minimised at buffer = 1.0, where the processor is strong enough to execute the sampled workload locally and cloud OpEx falls to zero. A useful implementation note is that the current final run becomes infeasible below buffer = 0.7 under the imposed latency-SLA budget, so the economically relevant region is the feasible operating band rather than the full theoretical buffer range.

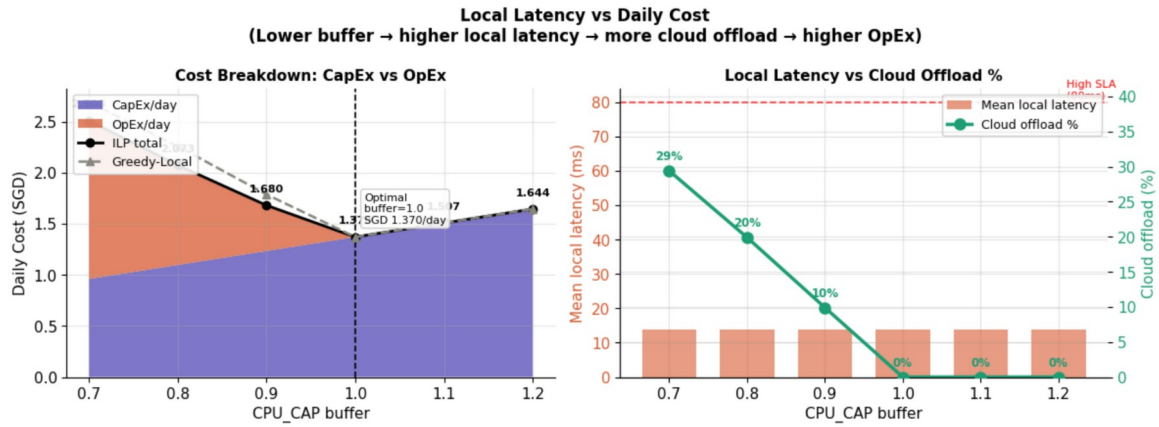


Figure 5.2. Plot linking CPU buffer, local latency, cloud offload percentage, and total daily cost

Finally, the economic significance becomes clearer at fleet scale. At the baseline buffer of 0.8, the ILP saves SGD 0.185 per vehicle per day relative to Greedy-Local. Although this appears modest for a single vehicle, it compounds to about SGD 675,000 over ten years for a fleet of 1,000 vehicles. This makes the optimisation layer commercially attractive because it improves cost efficiency through software logic rather than through additional sensors, hardware redesign or changes to the vehicle's physical architecture.

Implementation Feasibility

From an implementation perspective, the proposed model is relatively feasible because most of the required inputs already exist in standard autonomous-vehicle telemetry. The slides and notebook use CPU availability, network latency, task size, signal strength, task priority and weather-related information, all of which are quantities that can be measured or proxied in existing vehicle and communications stacks. This means the main implementation challenge is not data collection, but system integration and validation. For an initial pilot, the model can therefore be deployed as a decision-support layer running beside the current rule-based policy, allowing the organisation to compare recommended allocations against current practice before enabling live execution.

The main stakeholders are the vehicle manufacturer or AV platform owner, safety engineers, cloud or IT operations teams, and fleet operations managers. The manufacturer benefits from lower lifetime processing cost and more disciplined hardware sizing decisions. Safety engineers are responsible for setting the force_local logic, SLA thresholds and violation budget, since these directly affect whether a cloud assignment is acceptable. Cloud operations teams maintain the pricing assumptions and network telemetry feeds, while fleet operations teams monitor whether live decisions continue to meet latency expectations in actual driving conditions. Passengers and end users benefit indirectly because better task allocation supports both system responsiveness and operational reliability.

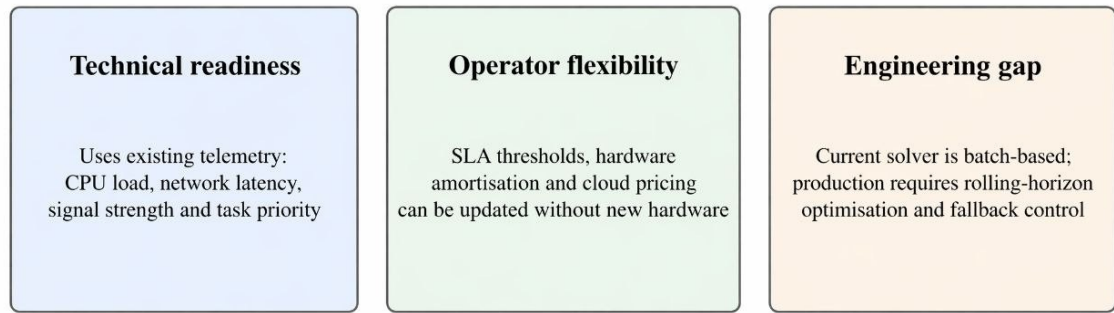


Figure 6.1. Feasibility summary based on technical readiness, operator flexibility and engineering gap

Several risks must still be controlled before production rollout. First, the present model is a static batch optimiser over an 800-task window, while real AV systems receive tasks sequentially. A production implementation should therefore move to a rolling-horizon scheduler that re-optimises continuously as new tasks arrive. Second, task priority is present in the data but not yet weighted in the objective function, so the production version should explicitly favour safety-critical tasks whenever trade-offs arise. Third, the current safety rule was triggered mainly by weak signal, whereas compound stress situations such as simultaneous poor signal, bad weather and low battery were not strongly represented in the sample. To control these risks, the system should include a conservative fallback policy, such as reverting to Greedy-Local whenever telemetry is missing the solver fails, or the operating environment falls outside the validated range.

Stage	Scope	Main owner	Monitoring metrics	Expected result
1. Offline shadow mode	Run ILP on historical or live-stream telemetry without executing its decisions.	Analytics + safety team	Daily cost/vehicle, cloud offload %, SLA breaches, force_local compliance	Verify that savings can be reproduced without worsening safety performance
2. Controlled pilot	Deploy on a small fleet or limited route segment for selected task classes.	Fleet ops + platform team	Latency by priority tier, fallback rate, solver availability, cloud budget	Demonstrate stable real-world behaviour and operational controllability
3. Production rollout	Embed a rolling-horizon optimiser inside the orchestration layer with automatic fallback.	Platform engineering	Sustained cost reduction, zero safety-rule violations, predictable cloud usage	Operationalise the model as a reliable dispatch policy

Table 6.1. Recommended phased rollout plan, ownership structure and success measures.

Overall, implementation feasibility is high for a pilot and moderate for full production. It is high for a pilot because the solution uses telemetry that is already available and does not require any new sensors or changes to the vehicle’s physical architecture. It is moderate for full production because software engineering work is still needed to make the optimiser dynamic, robust to missing data and validated under worst-case conditions. Nevertheless, the remaining gap is primarily one of orchestration and assurance rather than one of model design, which means the proposed solution is realistic, actionable and well aligned with the project rubric’s requirement for practical, monitorable implementation.

Outcome Evaluation

The project set out to determine whether a mathematical optimization approach could produce better task offloading decisions than simple heuristic rules in an autonomous vehicle setting. The results confirm that it can. The ILP consistently identifies assignments that neither baseline can replicate, because it evaluates all tasks jointly against the capacity and safety constraints rather than making isolated greedy choices. This joint reasoning is what allows it to reserve scarce local capacity for tasks where local execution is genuinely cheaper, while selectively offloading others where cloud latency is favourable.

An important observation is that the benefit of optimization is not uniform. The sensitivity analysis shows that when local capacity is generous (buffer = 1.0), the gap between the ILP and Greedy-Local narrows considerably, since most tasks can simply run locally without contention. The optimization advantage is greatest when resources are constrained (buffer = 0.5–0.7), which is precisely the operating region where real autonomous vehicles are most likely to find themselves during sustained high-load driving. This suggests the model adds the most value in the conditions that matter most.

Looking ahead, further work is needed to validate these findings under more demanding conditions and with richer model features. The specific limitations and corresponding recommendations are outlined in the sections that follow.

Limitations

Static single-period formulation. The model solves one batch of 800 tasks as a single decision window. In reality, tasks arrive continuously and assigning one task locally reduces the remaining capacity for the next. A static formulation cannot capture this sequential dependency, and may overestimate feasibility by assuming all capacity decisions are made simultaneously.

Unused priority information. The dataset contains a TaskPriority field (Low, Medium, High), yet this field does not enter the objective function. The optimizer therefore has no mechanism to preferentially protect high-priority tasks such as collision risk assessment. All latency reductions are valued equally regardless of task criticality.

Lightly exercised safety constraint. The force_local flag triggered on only 10.8% of tasks, driven almost entirely by weak signal strength (10.7%). Storm weather had a 0% occurrence rate and critically low battery affected just 0.1% of tasks. The safety constraint is correctly formulated, but it has not been tested under the compound adverse conditions (simultaneous low battery, weak signal, and storm) where its behaviour matters most.

Narrow sensitivity scope. Only the local CPU capacity buffer was varied. Sensitivity to network latency levels, the battery safety threshold, and the signal strength cutoff was not examined. This leaves open questions about which input parameters most influence the optimal solution and where estimation errors pose the greatest risk.

Recommendations

- Introduce priority weights into the objective (e.g., High = 3, Medium = 2, Low = 1) so the optimizer preferentially assigns fast processing to safety-critical tasks.
- Add a critical-task protection constraint that forces high-priority tasks to execute locally whenever cloud latency exceeds a defined threshold, providing an additional safety layer.

- Test under compound stress scenarios that combine heavy workload, degraded network quality, and harsh weather simultaneously, to validate the safety constraint under realistic worst-case conditions.
- Extend the formulation to a rolling-horizon approach that resolves over short look-ahead windows as new tasks arrive, allowing the model to adapt to real-time changes in conditions.
- Broaden the sensitivity analysis to cover network latency multipliers, battery safety thresholds, and signal strength cutoffs, producing a more complete robustness profile.
- Validate results on real-world telemetry from instrumented vehicle test platforms to confirm that the simulation-derived parameter distributions transfer to physical systems.

Conclusion

This project models the autonomous vehicle task offloading problem as a binary integer linear program and shows that the optimized approach consistently performs better than heuristic baselines across a range of operating conditions. The model does this by jointly evaluating all task assignments while accounting for safety and capacity constraints, something greedy methods cannot achieve. The advantage is especially evident in constrained environments, which are often the most relevant in practice.

At the same time, despite its effectiveness, the current model is based on a static, single-period design. It does not use available task priority information and has so far only been tested on simulated data where the safety constraint was only partially active. Closing these gaps through priority-weighted objectives, rolling-horizon scheduling, compound scenario testing, and real-world validation would greatly help to move the framework from proof of concept to operational readiness.

Overall, this project shows that prescriptive analytics provides a sound and effective way to manage safety-aware and cost-efficient resource allocation in real-world autonomous vehicle systems. The findings would be valuable for developing scalable and economically sustainable real-world driving solutions.

References

- Aleksandra. (2026, March 16). *Local vs Cloud Data Processing: Security Comparison*. DEV Community. https://dev.to/aleksandra_6bf530a531488b/local-vs-cloud-data-processing-security-comparison-5300
- Fortune Business Insights. (2026, March 30). *Autonomous Cars Market Size, Share & Growth Report [2034]*. <https://www.fortunebusinessinsights.com/industry-reports/autonomous-cars-market-100141>
- Viktor, P., & Kiss, G. (2026). Sensors in Self-Driving Vehicles: A Detailed Literature Review and New Trends. *Sensors*, 26(7), 2153. <https://doi.org/10.3390/s26072153>
- Qayyum, T. (2025). Vehicular Simulation Dataset (VEINS OMNeT++). Kaggle. <https://www.kaggle.com/datasets/ranatariq09/vehicular-simulation-dataset-veins-omnet>